WORKSHOP: PROGRAMMING LAB

› Session 2

Variables, types, expressions and input

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27

UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

RECAP

Last time

- Three ways to run Python: prompt, script, notebook
- The terminal: where am I, and where is my file?
- Error messages: last line first
- assistant.py v0.1 — correct, and badly built

```
print("Lecture hours:", 5 * 4)
print("Self-study:", 5 * 4 * 1.5)
print("Total:", 5 * 4 + 5 * 4 * 1.5)
print("Per day:", (5 * 4 + 5 * 4 * 1.5) / 7)
```

5 * 4 appears five times. Next week has 6 lectures: five edits, and one missed edit gives a wrong report. Today: give the number a name.

THE PLAN

Who teaches what

1 – 2

Ramon Winterhalder

Setup, first programs, errors · variables, types, input

3 – 4

Fatme Hachem

Conditions, logic, try / except · loops and lists

5

Ramon Winterhalder

Strings, tuples, sets

6 – 9

Fatme Hachem

Dictionaries, functions, files, libraries

Same lab, same rules: short intro, then exercises, then solutions together. Material for sessions 1–2 is on Ariel and GitHub.

MATERIALS

Where to find the material

myAriel

The official course page: announcements and all material, sessions 1–9.

myariel.unimi.it/user/index.php?id=14344

GitHub

Slides, exercise notebooks and solutions for sessions 1–3. No login needed.

github.com/ramonpeter/programming-lab-hcai-2026

No access to Ariel yet? Use GitHub. Everything for sessions 1-2 & 5 is there.

SESSION 2 · 4 HOURS

Today

1	**Names and values**	variables, naming rules, what a notebook remembers	40′
2	**Types**	int, float, str, bool · converting between them	45′
	Break		15′
3	**Expressions**	precedence, // and %, += and friends	30′
4	**Asking the user**	input() · choosing test inputs	45′
5	**Mini-project**	assistant.py v0.2 — and what you will add yourself	50′

01

Names and values

A variable is a name attached to a value — not a box that remembers how it was filled

ASSIGNMENT

= means: compute, then attach the name

```
a = 4
b = a * 2
a = 10
```



Right-hand side first. a * 2 was computed once, gave 8, and b was attached to 8. Moving a to 10 later changes nothing about b.

So x = x + 1 is not a contradiction: take the current x, add 1, attach x to the result.

NAMING

Naming rules — and one trap

Allowed		SyntaxError	
`total`	ok	`2nd_place`	starts with a digit
`_hidden`	ok	`first-name`	first minus name
`MAX_SIZE`	ok — constant	`my name`	space
`Class`	ok — not class	`class`	keyword

Allowed, and a disaster

```
print = 5
print("hello")
TypeError: 'int' object is not
callable
```

print is not a keyword, just a name Python gives you. Same for input, sum, max, list, str. Never use them for your own values.

Good names say what the value is: lecture_hours, not x or lh. Lowercase with underscores; ALL_CAPS for constants.

The notebook remembers

```
[1]  price = 10
```

```
[4]  price = price * 2
```

```
[5]  print(price)
```

[ ] shows the order cells ran, not their position.

- Names stay until you restart — even if you delete the cell
- Run a cell twice: it runs twice (price doubles again)
- After a restart, nothing exists until a cell creates it

Before you share a notebook: Restart → Run All. If it only works when clicked in one special order, it is broken. A .py script always starts from nothing.

EXERCISE A ⏱ 25 min · notebook lab02-exercises.ipynb

A1	core	Trace a, b and c line by line in a table, then run and check.
A2	core	Nine names: allowed or not? Test every one by assigning to it.
A3	core	print = 5, then print("hello"). Why? Get print back by restarting the kernel.
A4	core	Three cells, different run orders, then a restart. What is printed each time?
A5	challenge	counter = counter + 1 on a fresh kernel · w, h = 3, 4 · x = y = 0

A3 is not optional: until you restart, every later cell that uses print fails.

02

Types

Every value has a type, and the type decides what + means

TYPES

Four types for now

int

whole numbers

```
23
-4
0
```

float

decimal numbers

```
4.5
2.0
1e3
```

str

text, in quotes

```
"Ada"
"23"
""
```

bool

yes / no

```
True
False
3 > 2
```

```
>>> type("23")
<class 'str'>
>>> type(23 / 1)
<class 'float'>
```

type(x) tells you. Quotes win: "23" is text, even though it looks like a number. / always gives a float; int with float gives a float.

CONVERSION

Converting — when it can

`int("42")`	`42`	
`float("3.5")`	`3.5`	
`str(12) + "%"`	`'12%'`	
`int(3.99)`	`3`	cuts off — use round()
`int("3.5")`	`ValueError`	not whole-number text
`bool("False")`	`True`	any non-empty text

Two errors, two meanings

TypeError: the operation does not work for this type — "a" + 1

ValueError: right type, impossible content — int("twenty")

EXERCISE B ⏱ 30 min · notebook lab02-exercises.ipynb

B1 core Ten expressions: write the type you expect, then check with type().

B2 core Nine conversions, one per cell: predict the result or the error.

B3 core print("Price: " + price + " EUR") crashes. Fix it two different ways.

B4 stretch True + True, int(True), int(" 42 "), float("1e3") — explain.

B5 challenge round(0.5), round(1.5), round(2.5), round(3.5): find the rule in the docs.

Predict first, then run. The ones you got wrong are the ones to bring to the discussion.

03

Expressions

Who goes first — and a shorter way to update a name

Who goes first

`( )`	brackets
`**`	power — right to left
`* / // %`	left to right
`+ -`	left to right

```
print(-2 ** 2)
-4
print(10 - 4 - 3)
3
```

Update a name in place

```
balance = 100
balance -= 30  # balance = balance - 30
balance *= 2   # 140
balance //= 3  # 46
```

-2 ** 2 is -4: the minus sign is applied after the power. When in doubt, add brackets — they cost nothing and make it readable.

EXERCISE C ⏱ 20 min · notebook lab02-exercises.ipynb

C1 core Seven expressions: predict, then run. Watch -2 ** 2 and 17 // 5 * 5 + 17 % 5.

C2 core Follow balance through five updates.

C3 core Pizza for 23 people, 3 slices each, 8 per pizza: slices, whole pizzas (round up!), leftovers.

C4 stretch Final mark: 60% written, 40% project — weights as constants.

Use names for every number in C3 — then change people and run again.

04

Asking the user

input() always gives you text — and users type surprising things

INPUT

input() always returns text

```
age = input("How old are you? ")
print(age + 1)

$ python3 age.py
How old are you? 20
TypeError: can only concatenate str (not "int") to str
```

Before the fix

"20" is text. Text + number: TypeError.

```
age = int(input("How old are you? "))
print("Next year:", age + 1)

$ python3 age.py
How old are you? twenty
ValueError: invalid literal for int() with base 10:
'twenty'
```

After the fix

Works for 20. For "twenty" the conversion itself fails: ValueError. Handling that is session 3.

Test inputs that attack

bill.py — total, people, tip %

kind	input	result	what it tells you
normal	60, 4, 0	15.0	easy to check in your head
edge	10, 3, 0	3.33	three pay 9.99 — who pays the cent?
attack	50, 0, 10	ZeroDivisionError	nobody to split between
attack	50, two, 10	ValueError	text where a number was meant

Write the expected result before you run.

A test you only check afterwards always passes. Keep the crashes on your list — after session 3 you can fix them.

EXERCISE D ⏱ 30 min · notebook lab02-exercises.ipynb

D1	core	age.py: input, crash, fix with int(). Then type twenty: which error, why a different one?
D2	core	bill.py: total, people, tip % → amount per person, rounded to 2 decimals.
D3	core	Three test inputs for bill.py with expected results — written before running.
D4	stretch	Kitchen converter: grams → cups of flour (125 g) and sugar (200 g).
D5	challenge	How many more 4.2 MB photos fit on a 128 GB phone (14 GB system)?

Scripts today: lab02/age.py, lab02/bill.py … run them with python3 bill.py (Windows: python).

05

assistant.py v0.2

Every number stored once, and the user supplies them

MINI-PROJECT

From v0.1 to v0.2

```
SELF_STUDY_FACTOR = 1.5      # constant, at the top

name = input("What is your name? ")
lectures = int(input("How many lectures this week? "))
hours_per_lecture = float(input(
    "How many hours is one lecture? "))

lecture_hours = lectures * hours_per_lecture
self_study_hours = lecture_hours * SELF_STUDY_FACTOR
```

- Constants at the top, in CAPS
- Ask, convert, store — once
- Compute each number once, in a name
- Print the report from the names

Stretch: f-strings.
f"Hours per day: {hours_per_day:.1f}" puts the value into the text, with one decimal.

EXERCISE E ⏱ 35 min · notebook lab02-exercises.ipynb

E1 `core` Copy assistant.py to lab02. Constant at the top, ask for name, lectures and hours per lecture, compute once, print the report.

E2 `stretch` Rewrite the report lines with f-strings and :.1f.

E3 `challenge` Ask for study days and hours of sleep: hours per study day, and % of waking time.

```
==================================
   STUDY ASSISTANT  v0.2
==================================
What is your name? Ada
How many lectures this week? 6
How many hours is one lecture? 4

Hello, Ada! Your week:
Lecture hours: 24.0
Self-study hours (1.5 per
  lecture hour): 36.0
Total hours: 60.0
Hours per day (7 days): 8.6
==================================
```

AFTER SESSION 3

v0.3 is yours

What is missing is exactly what session 3 with Fatme Hachem teaches: decisions (if / elif / else, and / or) and surviving bad input (try / except). Afterwards, add these rules yourself:

Advice

more than 8 h per day: warn · fewer than 3: room for more · otherwise: realistic

Bad input

someone types four instead of 4: a friendly message, not a crash

Impossible input

0 study days, negative lectures: say so, instead of dividing by zero

Practice, nothing to hand in. Start with the test inputs from D3 — they are the cases v0.3 has to survive.

Today you…

- gave values names, and saw that = attaches, not equates
- met four types and converted between them
- used input() — and found the inputs that break it
- built assistant.py v0.2

Next · session 3
with Fatme Hachem: conditions, logical operators, try / except.

Project route? Decide by session 3.

Keep assistant.py — it is your starting point for v0.3.